

Assignment 5 solution

1. 【考点：排序基本概念】关于排序算法的“稳定性（Stability）”，下列说法中最为准确且严谨的是（ ）
- A. 某种排序算法被判定为“不稳定”，意味着对于任意输入序列，该算法都会改变相同关键字元素的相对前后顺序
 - B. 在基于比较的内部排序算法中，只要该算法的最坏时间复杂度达到了 $O(n \log_2 n)$ ，它就必然是一种不稳定的排序算法
 - C. 稳定排序算法在排序过程中，绝对不会发生跨越多个元素的“长距离”交换操作
 - D. 强制将一个不稳定的排序算法改为稳定排序，其空间复杂度必然要上升到 $O(n)$

正确答案：C

详细解析：

C 正确：不稳定的根本原因往往是因为算法中存在“跳跃式”或“长距离”的交换或移动（例如快排的左右指针交换、希尔排序的跨步长插入、堆排序的堆顶与堆底交换），这极易破坏相同关键字的相对顺序。而经典的稳定排序（如直接插入、冒泡、归并）每次都只在相邻或局部进行比较和移动

A 错误：不稳定是指“可能”改变相对顺序，存在某些特定序列排序后相对顺序恰好没变的特例

B 错误：二路归并排序的最坏时间复杂度是 $O(n \log_2 n)$ ，但它是严格稳定的

D 错误：要使不稳定排序变稳定，可以通过给每个元素增加一个初始“序号（索引）”作为第二关键字，空间复杂度取决于数据项本身体积，并不是所有算法都需要额外 $O(n)$ 辅助空间

2. 【考点：插入排序】对一个包含 n 个元素的无序序列进行折半插入排序。与传统的直接插入排序相比，折半插入排序在执行全过程中（ ）
- A. 极大地减少了元素的移动次数，但关键字的比较次数保持不变
 - B. 元素的移动次数没有减少，但关键字的比较次数大幅减少，且该比较次数与初始序列的排列状态基本无关
 - C. 无论是关键字的比较次数还是元素的移动次数，都得到了显著的减少
 - D. 当初始序列已经处于严格升序（正序）状态时，折半插入排序的关键字比较次数少于直接插入排序

正确答案：B

详细解析：

折半插入排序的优化核心在于：利用折半查找（二分查找）来确定新元素在已基本有序的子序列中的**插入位置**

因为二分查找确定位置的比较次数固定为 $O(\log_2 n)$ 级别，与初始序列是有序还是无序无关

但是，一旦位置确定后，将后续元素向后挪动腾出空位的物理操作并没有减少，最坏和平均情况下的元素移动次数依然是 $O(n^2)$

特别注意 **D 错误：**当初始序列完全正序时，直接插入排序每趟只需比较 1 次，总比较次数仅为 $n-1$ （即 $O(n)$ ）；而折半插入排序依然要老老实实跑完二分查找，比较次数仍为 $O(n \log_2 n)$

3. 【考点：插入排序】已知一个初始关键字序列为 {50, 26, 38, 80, 70, 56, 12, 40}。现采用希尔排序（Shell Sort）对其进行升序排列，若第一趟排序选定的增量（步长） $d_1 = 4$ ，则第一趟希尔排序彻底结束后的关键字序列是（ ）
- A. 12, 26, 38, 40, 50, 56, 70, 80
 - B. 26, 38, 50, 80, 12, 40, 56, 70
 - C. 50, 26, 12, 40, 70, 56, 38, 80
 - D. 40, 26, 12, 50, 70, 56, 38, 80

正确答案：C

详细解析：

增量 $d_1 = 4$ ，意味着序列被拆分为 4 个独立的子序列，每个子序列内部进行直接插入排序：

子序列 1（下标 0, 4）：{50, 70} → 排序后保持 {50, 70}

子序列 2（下标 1, 5）：{26, 56} → 排序后保持 {26, 56}

子序列 3（下标 2, 6）：{38, 12} → 排序后变为 {12, 38}

子序列 4（下标 3, 7）：{80, 40} → 排序后变为 {40, 80}

将各子序列组合回原数组对应的位置：

下标 0 是 50，下标 1 是 26，下标 2 是 12，下标 3 是 40；

下标 4 是 70，下标 5 是 56，下标 6 是 38，下标 7 是 80

最终序列为：{50, 26, 12, 40, 70, 56, 38, 80}，对应 C 选项

4. 【考点：交换/选择/堆排序】在数据结构常用的内部排序算法中，往往有些算法具备特殊的工程性质。下列五种排序算法中：

① 直接插入排序 ② 简单选择排序 ③ 起泡排序 ④ 快速排序 ⑤ 堆排序

具有“在整个排序的任何一趟彻底结束后，都至少能保证有一个元素被放置到了其最终排序后的绝对正确位置上”这一特征的算法组合是（ ）

A. 仅 ②③④

B. 仅 ②④⑤

C. 仅 ②③④⑤

D. ①②③④⑤ 全都是

正确答案：C

详细解析：

② **简单选择排序**：每趟都会选出一个剩余元素中的最小值/最大值，放到它最终的位置上

③ **起泡排序**：每趟通过相邻交换，把当前剩余元素的最大值/最小值像气泡一样“浮”到最终的边缘位置

④ **快速排序**：每次 Partition（划分）结束后，选定的枢轴（Pivot）元素就会被精准卡在左右子区间正中间，这个位置就是它最终归宿

⑤ **堆排序**：初始建堆后，每趟将堆顶最大值（大根堆）与当前堆底元素交换，并从堆中剥离。这个被剥离交换到尾部的元素就是其最终正确位置

① **直接插入排序**：完全不具备此性质。插入排序只是保证前 i 个元素“相对有序”，由于后面还可能更小的元素插到前面，导致当前看似排好序的元素整体向后大漂移，因此无法保证任何元素的绝对最终位置

5. 【考点：交换排序】快速排序的时间性能在极大程度上取决于划分（Partition）的对称性。设有一个包含 n 个不同元素的无序序列，若每次划分选取的枢轴（Pivot）元素都恰好是当前待排序列中的中位数，则快速排序的空间复杂度为（ ）

A. $O(1)$

B. $O(\log_2 n)$

C. $O(n)$

D. $O(n \log_2 n)$

正确答案：B

详细解析：

快速排序的**空间复杂度**主要开销来源于**递归调用栈的深度**

当每次划分的枢轴都恰好是中位数时，递归树是一棵形态极其完美的平衡二叉树，树的高度为 $\log_2 n$ 。此时，递归栈的最大深度也达到了理想的最小值 $\lceil \log_2 (n+1) \rceil$ 。因此，最好情况下的空间复杂度为 $O(\log_2 n)$ 。（注意：如果划分极度不对称，退化为单链表，最坏空间复杂度会变成 $O(n)$ ）

6. 【考点：选择排序】对一组初始关键字序列 {15, 25, 40, 30, 10, 50} 构建初始大根堆（采用顺序存储，一维数组下标从 1 开始）。在自底向上的初次建堆调整彻底完成后，该一维数组中排在第四个位置（即数组下标为 4）的元素是（ ）

- A. 15
- B. 25
- C. 30
- D. 40

正确答案：B

详细解析：

初始一维数组状态：[1]:15, [2]:25, [3]:40, [4]:30, [5]:10, [6]:50。总长度 $n=6$
非叶子结点下标分别为 $\lfloor 6/2 \rfloor = 3, 2, 1$ 。按 $3 \rightarrow 2 \rightarrow 1$ 的顺序自底向上进行下滤调整：

调整结点 3 ([3]:40)：其左孩子为 [6]:50。因为 $50 > 40$ ，交换两者。数组变为：

[1]:15, [2]:25, [3]:50, [4]:30, [5]:10, [6]:40

调整结点 2 ([2]:25)：其左孩子 [4]:30，右孩子 [5]:10。较大孩子是 30。因为 $30 > 25$ ，交换两者。数组变为：[1]:15, [2]:30, [3]:50, [4]:25, [5]:10, [6]:40

调整结点 1 ([1]:15)：其左孩子 [2]:30，右孩子 [3]:50。较大孩子是 50。因为 $50 > 15$ ，交换两者。数组变为：[1]:50, [2]:30, [3]:15, [4]:25, [5]:10, [6]:40

此时 15 被换到了 [3]，需对 [3]:15 继续向下筛：其左孩子为 [6]:40。因为 $40 > 15$ ，再次交换。数组最终定格为：[1]:50, [2]:30, [3]:40, [4]:25, [5]:10, [6]:15

查看数组下标为 4 的位置，存放的是 **25**

7. 【考点：归并排序】在对包含 n 个元素的序列进行标准的递归型二路归并排序 (Merge Sort) 时，系统的算法总空间复杂度被定义为 $O(n)$ 。关于这一空间复杂度的微观来源，下列说法中正确的是 ()
- A. 它完全来自于递归调用时由操作系统分配的栈空间
 - B. 它完全来自于为了将两个有序子表合并为一个有序表而强制开辟的辅助数组空间
 - C. 它是递归调用栈空间 $O(\log_2 n)$ 与合并辅助数组空间 $O(n)$ 的代数加和，根据渐进复杂度取大者法则最终记为 $O(n)$
 - D. 如果将递归实现改为非递归 (迭代) 实现，归并排序的空间复杂度可以成功降至 $O(1)$

正确答案：C

详细解析：

递归版的归并排序在运行期间同时占用了两种空间：一是递归调用的物理工作栈，其最大深度等于归并树的高度 $O(\log_2 n)$ ；二是在动态执行 Merge 操作时，必须申请一个与原数组等大的辅助转存数组，空间大小为 $O(n)$

总空间复杂度 = $O(n) + O(\log_2 n) = O(n)$

即使你用迭代 (非递归) 方法消除了递归栈，由于合并两个有序序列时“鸠占鹊巢”的物理限制，依然无法逃掉 $O(n)$ 辅助数组的开辟，因此非递归版的空间复杂度依然是 $O(n)$ ，D 选项错误

8. 【考点：基数排序】已知一组包含 1000 个随机无序的三位十进制正整数 (例如 342, 105 等)，现采用最低位优先 (LSD) 的基数排序 (Radix Sort) 对其进行升序排列。在排序的全过程中，该算法一共需要进行的分配 (Distribute) 和收集 (Collect) 的总趟数分别是 ()
- A. 1 趟分配，1 趟收集
 - B. 3 趟分配，3 趟收集
 - C. 10 趟分配，10 趟收集
 - D. 3 趟分配，10 趟收集

正确答案：B

详细解析：

基数排序的趟数**仅仅取决于关键字的位数 (属性个数)**

本题的目标数据是“三位十进制数”，这意味着每个数据由 3 个数位 (个位、十位、百位) 决定，即关键字基数分解后的维度 $d = 3$

算法会严格按照：个位分配与收集 (第 1 趟) $\rightarrow$ 十位分配与收集 (第 2 趟) $\rightarrow$ 百位分配与收集 (第 3 趟) 的流程推进。因此共执行 3 趟分配和 3 趟收集。整个过程与数据总量 (1000 个) 无关

9. 【考点：外部排序】在对海量文件进行外部排序（External Sorting）的优化过程中，理论上增加归并路数 m 可以有效减少归并的总趟数，从而大幅削减其耗时的磁盘 I/O 次数。然而，在实际工程实现中，归并路数 m 绝对不是越大越好，盲目增大 m 带来的最大副作用是（ ）
- A. 会导致第一阶段生成的初始归并段的长度被迫大幅度缩短
 - B. 会导致内部归并时在 m 个归并段中选择最小关键字的比较次数激增，使得 CPU 运算时间反演为系统瓶颈
 - C. 会使得最佳归并树的高度不降反升，直接引发磁盘 I/O 次数不合常理地增加
 - D. 外部排序的空间复杂度会随着 m 的增大而呈现指数级的恶性上升

正确答案：B

详细解析：

外部排序的总时间 = 内部常规排序时间 + 外部磁盘 I/O 时间 + **内部归并所需 CPU 时间**

当我们把归并路数 m 增大时，外部归并树变矮，磁盘 I/O 确实减少了。但是，在内部要把 m 个段的段头元素挑出一个最小的送进缓冲区，传统的两两比较需要 $m-1$ 次比较。如果不引入高级结构（如败者树 Loser Tree），随着 m 的盲目增大，CPU 在这里浪费的比较时间会呈线性爆发，导致 CPU 运算时间取代磁盘 I/O 成为系统新的严重卡顿瓶颈

10. 【考点：排序综合】在数据结构所有主流内部排序算法中，某种算法在最坏情况下的时间复杂度依然能够稳定在 $O(n \log_2 n)$ ，同时它是一种不稳定的排序算法，并且其辅助空间复杂度为 $O(1)$ 。具备上述所有核心物理特征的算法是（ ）
- A. 快速排序（Quick Sort）
 - B. 希尔排序（Shell Sort）
 - C. 堆排序（Heap Sort）
 - D. 归并排序（Merge Sort）

正确答案：C

详细解析：

A 排除：快速排序在最坏情况（初始有序）下会恶化退化为 $O(n^2)$ ，且最坏空间复杂度退化为 $O(n)$

B 排除：希尔排序最坏时间复杂度根据增量不同在 $O(n^2)$ 或 $O(n^{1.5})$ 左右，达不到 $O(n \log_2 n)$

D 排除：归并排序虽然最坏能保持 $O(n \log_2 n)$ 且稳定，但其空间复杂度高达 $O(n)$

C 完全吻合：堆排序凭借完全二叉树的物理树形调整，无论初始序列如何恶劣，其建堆和每趟筛选的最大时间雷打不动地卡死在 $O(n \log_2 n)$ ；同时由于直接在原一维数组内进行就地指针交换，空间复杂度为极佳的 $O(1)$ ，且为典型的不稳定排序

11. 【考点：快速排序】在为华为昇腾（Ascend）NPU 开发底层的向量化排序算子时，工程师采用快速排序（Quick Sort）对海量的特征张量进行原地排序。为了极致压榨硬件性能，该算子在 NPU 核心极其有限的局部内存（Local Memory）中分配了固定大小的函数调用栈。在运行某一批“特征极其稀疏且存在大量连续重复零值”的张量数据时，NPU 核心突然发生 **Stack overflow**（栈溢出）崩溃。导致该崩溃的最本质原因是（ ）
- A. 快速排序是不稳定的排序算法，大量重复的零值导致元素位置频繁交换，引发内存越界
 - B. 当数组元素基本有序或存在大量重复元素时，若未优化枢轴（Pivot）选择，快速排序的递归树将严重向一侧倾斜，导致空间复杂度退化为 $O(n)$ ，撑爆有限的硬件栈
 - C. 快速排序在处理大量零值时，其时间复杂度会退化为 $O(n^2)$ ，导致 NPU 看门狗超时复位
 - D. 在原生的快速排序中，Partition 划分过程需要额外申请 $O(n)$ 的辅助数组来暂存数据，导致局部内存耗尽

正确答案：B

详细解析：

考查**快速排序的空间复杂度与初始状态的极度敏感性**

A 选项错误：稳定性指的是相等元素的相对顺序是否改变，与内存越界或栈溢出毫无关联

C 选项错误：时间复杂度退化为 $O(n^2)$ 确实会发生（导致变慢），但这是“超时”的原因，并非题目中明确指出的 **Stack overflow**（栈溢出）的直接原因

D 选项错误：快速排序是**原地排序**，Partition 过程通过双指针交换元素，空间复杂度为 $O(1)$ ，不需要申请 $O(n)$ 的辅助数组

B 选项正确：快速排序的空间复杂度完全取决于**递归调用栈的深度**。最好情况下递归树平衡，深度为 $O(\log_2 n)$ ；但在处理极其不平衡的划分（如全等数组或已有序数组，且仅选第一个元素为枢轴）时，递归树退化成单链表，深度暴增至 $O(n)$ 。在 NPU 这种栈空间极其金贵的底层硬件中，必然导致栈溢出

12. 【考点：堆排序】在大语言模型（如 DeepSeek-V3）的混合专家（MoE）架构中，路由网络需要为当前输入的 Token 从 256 个备选专家网络中，快速挑选出打分最高（概率最大）的前 4 个专家（即 Top-4 问题）

为了将算法的时间开销降到最低，系统底层的 C++ 引擎决定采用堆（Heap）这一数据结构来优化查找过程。关于该场景下的堆操作，以下在理论上最优的设计是（ ）

A. 将 256 个专家的打分构建成一个包含 256 个元素的最大堆（Max-Heap），建堆的时间复杂度为 $O(n \log n)$ ，随后执行 4 次输出堆顶并调整的操作

B. 维护一个容量仅为 4 的最小堆（Min-Heap）。遍历后续的 252 个专家打分，若大于堆顶元素，则替换堆顶并进行“向下调整”。最终堆中保留的就是 Top-4

C. 将 256 个专家的打分构建成一个包含 256 个元素的最小堆（Min-Heap），然后直接取堆底的 4 个叶子节点即可

D. 采用二路归并排序对 256 个专家进行完全排序，因为归并排序的比较次数最少，寻找 Top-4 的整体时间复杂度最低

正确答案：B

详细解析：

考查**堆排序的应用（Top-K 问题）及建堆的时间复杂度**

A 选项致命错误：包含 n 个元素的建堆过程，通过自底向上的“向下调整”，其时间复杂度是严格的 $O(n)$ ，而非 $O(n \log n)$ 。虽然建最大堆取前 4 个可行，但维护 256 大小的堆在缓存局部性上不如小堆

C 选项错误：堆只保证父子节点的大小关系，不保证同一层或叶子节点之间的全局有序性。最小堆的叶子节点不一定是最大的元素

D 选项错误：如果只需要 Top-K，进行全排序（如归并排序需要 $O(n \log n)$ ）纯属算力浪费

B 选项正确：找最大的 K 个元素，应维护容量为 K 的**最小堆**。堆顶是这 K 个数中最小的“守门员”。如果新来的数比守门员大，就踢掉守门员进堆并重新调整。这种方法在处理海量流式数据时（如 $N \gg K$ ）极大地节省了内存和比较次数，整体时间复杂度仅为 $O(N \log K)$

13. 【考点：外部排序】在分析数亿条出行记录时，需要将散落在几十个按月存储的 CSV 文件合并，并按（取车时间，经纬度）进行全局排序。由于数据量高达数百 GB，无法全部装入内存，分析师采用了多路归并的外部排序（External Sorting）
为了减少昂贵的磁盘读写次数，分析师决定将归并的路数 k 从 2 路大幅提升到 16 路。以下推论绝对正确的是（ ）

A. 增加归并路数 k 可以减少初始归并段的个数，从而直接降低总的磁盘 I/O 次数

B. 归并趟数 $S = \lceil \log_k r \rceil$ （ r 为初始段个数）。增大 k 会显著减少归并趟数，从而减少磁盘 I/O，但如果不引入“败者树（Loser Tree）”，内存中比较关键字的 CPU 耗时会大幅增加

C. 无论 k 增加到多大，由于每次归并都需要把数据读入内存再写回磁盘，总的磁盘 I/O 次数只与原始数据总量相关，与 k 无关

D. 在 16 路归并中，为了选出最小的记录，使用败者树进行比较的次数会多于传统的直接比较方法，但胜在结构稳定

正确答案：B

详细解析：

考查**外部排序的核心矛盾：磁盘 I/O 优化与内部比较的博弈**

A 选项错误：增加归并路数 k 并不能减少初始归并段的个数 r 。初始归并段的个数是由内存工作区的大小和生成初始段的算法（如置换-选择排序）决定的

C 选项错误：外部排序的磁盘 I/O 总次数 $\approx$ 每趟读写一遍数据 $\times$ (归并趟数 + 1)。趟数减少，I/O 必然减少

D 选项错误：在 k 路直接比较中，找最小值需要 $k-1$ 次比较。而使用败者树，由于是树形结构，找最小值仅需要 $\lceil \log_2 k \rceil$ 次比较，大大减少了内部比较次数

B 选项正确：增大 k 是减少归并趟数（进而减少 I/O）的最直接方法。但代价是，每输出一个元素，在内存中从 k 个候选者挑最小值的内部比较次数会增加，导致内部时间复杂度上升为 $O(n \log_2 k)$ 。必须引入败者树将内部比较复杂度降回与 k 无关的水平

14. 【考点：直接插入排序】在分布式共识算法（如 Raft）中，某个从节点（Follower）由于短暂的网络抖动，刚刚接收到主节点补发的一小批日志条目。此时，该从节点内存中的千万级日志数组绝大部分已经是严格按时间戳递增排序的，只有尾部新追加的几十条日志存在轻微的乱序。如果此时需要调用一个算法将整个日志数组彻底排序，在不考虑额外空间开销的前提下，以下排序算法中执行效率最高的是（ ）

- A. 直接插入排序（Straight Insertion Sort）
- B. 简单选择排序（Simple Selection Sort）
- C. 快速排序（Quick Sort）
- D. 二路归并排序（Merge Sort）

正确答案：A

详细解析：

考查**四大经典排序算法对“初始状态极度偏向有序”的敏感度**

题干关键词：“绝大部分已经严格排序，只有少量乱序”（即**基本有序**）

B 选项错误：简单选择排序极其死板，无论初始状态如何，找最小值的比较次数永远是 $(n(n-1))/2$ ，时间复杂度恒为 $O(n^2)$

C 选项错误：如果不对枢轴做“三数取中”等优化，快速排序面对基本有序的数组是灾难，会退化成 $O(n^2)$ 。即便优化，最好也就是 $O(n \log n)$

D 选项错误：归并排序同样对初始状态不敏感，不管你有序无序，无脑切分合并，时间复杂度恒定为 $O(n \log n)$

A 选项正确：直接插入排序在面对“基本有序”的序列时，具有降维打击的优势。当新元素向前比较时，一碰就发现已经比前面的大了，立刻停止比较原地坐下。其比较次数和移动次数都会大幅下降，时间复杂度从最坏的 $O(n^2)$ 骤降至接近 $O(n)$

15. 【考点：基数排序】币安（Binance）的后台安全系统需要对 2026 年上半年的海量数字货币交易流水进行排序审查。每条流水包含三个独立的整数维度：[年份 Year]（只有 2026）、[月份 Month]（1-6）、[交易额 Amount]（0-99999999 的整数）

现要求审查报表必须先按 Year 升序，如果 Year 相同则按 Month 升序，如果 Month 相同再按 Amount 升序

工程师决定采用最低位优先（LSD）的基数排序（Radix Sort）思想，通过调用底层的“桶排序（Bucket Sort）”模块进行多次分配与收集。关于该算法的设计，以下说法正确的是（ ）

- A. LSD 的执行顺序必须是：第 1 趟按 Year 桶排 → 第 2 趟按 Month 桶排 → 第 3 趟按 Amount 桶排
- B. 作为子过程的底层桶排序模块，必须使用稳定（Stable）的排序算法或分配收集机制，否则后续趟次的排序会破坏前一趟排好的内部顺序
- C. 由于 Amount 的取值范围极大（0-99999999），基数排序在这种场景下的时间复杂度必定退化为 $O(n^2)$ ，不如直接使用快速排序
- D. LSD 的执行顺序必须是：第 1 趟按 Amount 桶排 → 第 2 趟按 Month 桶排 → 第 3 趟按 Year 桶排，且子模块可以是不稳定排序

正确答案：B

详细解析：

考查**多关键字基数排序（LSD 与 MSD）的执行逻辑与稳定性刚需**

题目要求最终优先级：Year（第一主键）> Month（第二主键）> Amount（第三主键）

LSD（最低位优先）的核心逻辑是“逆练”：必须从最次要的关键字（最低位）开始排，最后排最主要的关键字（最高位）

因此，LSD 的正确趟次顺序是：第 1 趟 Amount → 第 2 趟 Month → 第 3 趟 Year。这样最后

一趟排完后, `Year` 整体有序, 而在 `Year` 相同的桶内, 保留着上一趟按 `Month` 排好的相对顺序。这直接排除了 **A 选项**

D 选项错误: 趟次是对的, 但**错在后半句**。如果底层的子排序是不稳定的, 当第 2 趟按 `Month` 把数据塞进“5月桶”时, 原本在第 1 趟排好的 `Amount` 的顺序就会被打乱!

C 选项错误: 基数排序的时间复杂度为 $O(d(n+r))$, d 是趟数, r 是基数。即便 `Amount` 很大, 只要按数位(个、十、百、千...)拆解, 仍然是线性的时间复杂度, 不会退化为 n^2

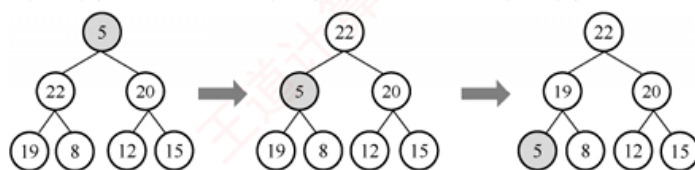
B 选项正确: 基数排序的灵魂就在于, 其内部依赖的每一次位排序(或子关键字排序), 必须是**绝对稳定的**。这是保证高位排序后, 低位信息不丢失的唯一数学前提

16. **20. A**

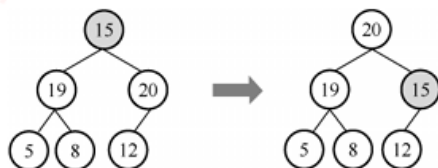
依题意, 对数组 M 进行第一趟快速排序后, 会将 M 划分为三部分: 小于枢轴值的块 P , 枢轴元素, 大于枢轴值的块 Q 。 P 和 Q 块内都是无序的, 但是 P 块内的数据 $\leq$ 枢轴值, Q 块内的数据 $\geq$ 枢轴值, 因此 P 块内的数据 $\leq Q$ 块内的数据, 即 P 与 Q 块间有序。

17. **21. B**

该序列已调整成大根堆, 接下来进行第一次删除操作: 删除 28 后, 将 5 放入堆顶; 然后自上而下调整。22 和 20 比较, 较大者 22 与 5 比较, 交换; 19 和 8 比较, 较大者 19 与 5 比较, 交换。



接下来进行第二次删除操作: 删除 22 后, 将 15 放入堆顶; 然后自上而下调整。19 和 20 比较, 较大者 20 与 15 比较, 交换; 15 直接与其仅有的左孩子 12 比较, 不交换。



因此, 进行两次删除操作后, 得到的新堆是 20, 19, 15, 5, 8, 12。

18. **20. C**

从左至右对这三个有序序列进行 2 路归并排序的过程如下。

① 合并(3, 5)和(7, 9): 比较 3 和 7 (1 次), 选择 3; 比较 5 和 7 (1 次), 选择 5; 剩余元素 7 和 9 直接加入。② 合并(3, 5, 7, 9)和(6): 比较 3 和 6 (1 次), 选择 3; 比较 5 和 6 (1 次), 选择 5; 比较 7 和 6 (1 次), 选择 6; 剩余元素 7 和 9 直接加入。总共比较 5 次。

19. **22. D**

简单选择排序每趟从未排序部分选出最小(或最大)元素, 与未排序部分的首元素交换。无论输入如何, 最多执行 $n-1$ 次交换, 总移动次数为 $O(n)$, 是所有选项中最少的。而起泡排序、直接插入排序和快速排序在最坏情况下均需 $O(n^2)$ 次移动(如逆序时起泡和插入需要大量移位, 快速递归分割导致频繁交换)。因此, 简单选择排序在最坏情况下的元素移动次数最少。

20. **23. A**

第 1 趟排序后, 序列呈现出以间隔 3 分组的局部有序特征: 下标模 3 相同的元素各自有序(如位置 1, 4, 7 上的 25, 10, 15 变为 10, 15, 25; 位置 2, 5, 8 上的 40, 20, 35 变为 20, 35, 40), 这是希尔排序在增量 $d=3$ 时对各子序列进行插入排序的典型特征。第 2 趟进一步调整, 符合增量缩小至 1 的结果。而其他三种排序方法均无法产生这种跨距离分组、局部有序的中间状态。

21. **16. D**

利用败者树归并升序归并段, 每次需要得到当前的最小关键字, 因此记录“冠军”的结点保存的只能是最小关键字或最小关键字所在的归并段号。通过败者树找出最小关键字后, 还需要找到该关键字所在的归并段, 并移动段内元素, 以继续比较下一个元素, 下面分别进行讨论: ① 假设记录“冠军”的结点保存的是最小关键字所在的归并段号, 则能直接得到最小关键字及其所在的段, 时间复杂度低。② 假设记录“冠军”的结点保存的是最小关键字, 则查找其所在的段需要检索所有段的首元素是否与该最小关键字相等, 时间复杂度高。为了提高效率, 记录“冠军”的结点保存的是最小关键字所在的归并段号, 而不是最小关键字。

22. cmpCountSort 算法基于计数排序的思想,对序列进行排序。cmpCountSort 算法遍历数组中的元素, count 数组记录比对应待排序数组元素下标大的元素个数,例如, count[1]=3 的意思是数组 a 中有三个元素比 a[1] 小,即 a[1] 是第四大元素, a[1] 的正确位置应是 b[3]。

1) 排序结果为 b[6]={-10,10,11,19,25,25}。

2) 由代码 for(i=0;i<n-1;i++) 和 for(j=i+1;j<n;j++) 可知,在循环过程中,每个元素都与它后面的所有元素比较一次(即所有元素都两两比较一次),比较次数之和为 $(n-1)+(n-2)+\cdots+1$,所以总比较次数是 $n(n-1)/2$ 。

3) 不是。需要将程序中的 if 语句修改如下:

```
if(a[i]<=a[j]) count[j]++;
else count[i]++;
```

若不加等号,两个相等的元素比较时,前面元素的 count 值会加 1,则导致原序列中靠前的元素在排序后的序列中处于靠后的位置。

23. 1) 算法思想。

【方法 1】定义含 10 个元素的数组 A,初始时元素值均为该数组类型能表示的最大数 MAX。

for M 中的每个元素 s

if (s<A[9]) 丢弃 A[9] 并将 s 按升序插入 A;

当数据全部扫描完毕,数组 A[0]~A[9] 保存的就是最小的 10 个数。

【方法 2】定义含 10 个元素的大根堆 H,元素值均为该堆元素类型能表示的最大数 MAX。

for M 中的每个元素 s

if (s<H 的堆顶元素) 删除堆顶元素并将 s 插入 H;

当数据全部扫描完毕,堆 H 中保存的就是最小的 10 个数。

2) 算法平均情况下的时间复杂度是 $O(n)$,空间复杂度是 $O(1)$ 。

24. 1) 当文件中的 n 个记录升序排列时,只生成一个归并段,长度达到最大,为 n。若初始工作区内的 m 个元素都大于输入文件中剩下的所有记录,则第一个归并段的长度就为 m,此时为第一个归并段长度最小的情况。排序过程如下表所示。

输出文件 FO	工作区 WA	输入文件 FI
—	—	51, 94, 37, 92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
—	51, 94, 37, 92	14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37	51, 94, 14, 92	63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51	94, 14, 92, 63	15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63	15, 94, 14, 92	99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92	15, 94, 14, 99	48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92, 94	15, 14, 99, 48	56, 23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92, 94, 99	15, 14, 56, 48	23, 60, 31, 17, 43, 8, 90, 166, 100
37, 51, 63, 92, 94, 99#	15, 14, 56, 48	23, 60, 31, 17, 43, 8, 90, 166, 100
14	15, 23, 56, 48	60, 31, 17, 43, 8, 90, 166, 100
14, 15	60, 23, 56, 48	31, 17, 43, 8, 90, 166, 100
14, 15, 23	60, 31, 56, 48	17, 43, 8, 90, 166, 100
14, 15, 23, 31	60, 17, 56, 48	43, 8, 90, 166, 100
14, 15, 23, 31, 48	60, 17, 56, 43	8, 90, 166, 100
14, 15, 23, 31, 48, 56	60, 17, 8, 43	90, 166, 100
14, 15, 23, 31, 48, 56, 60	90, 17, 8, 43	166, 100
14, 15, 23, 31, 48, 56, 60, 90	166, 17, 8, 43	100
14, 15, 23, 31, 48, 56, 60, 90, 166	100, 17, 8, 43	—
14, 15, 23, 31, 48, 56, 60, 90, 166#	100, 17, 8, 43	—
8	100, 17, 43	—
8, 17	100, 43	—
8, 17, 43	100	—
8, 17, 43, 100	—	—
8, 17, 43, 100#	—	—

生成三个初始归并段,分别是 37, 51, 63, 92, 94, 99; 14, 15, 23, 31, 48, 56, 60, 90, 166; 8, 17, 43, 100。

2) 最大值为 n, 最小值为 m。

25. 04.【解答】

1) 算法的基本设计思想

由题意可知, 将最小的 $\lfloor n/2 \rfloor$ 个元素放在 A_1 中, 其余的元素放在 A_2 中, 分组结果即可满足题目要求。仿照快速排序的思想, 基于枢轴将 n 个整数划分为两个子集。根据划分后枢轴所处的位置 i 分别处理:

- ① 若 $i = \lfloor n/2 \rfloor$, 则分组完成, 算法结束。
- ② 若 $i < \lfloor n/2 \rfloor$, 则枢轴及之前的所有元素均属于 A_1 , 继续对 i 之后的元素进行划分。
- ③ 若 $i > \lfloor n/2 \rfloor$, 则枢轴及之后的所有元素均属于 A_2 , 继续对 i 之前的元素进行划分。

基于该设计思想实现的算法, 无须对全部元素进行全排序, 其平均时间复杂度是 $O(n)$, 空间复杂度是 $O(1)$ 。

26. 【考点: 排序算法的分析和应用】关于各内部排序算法在特定数据状态下的表现, 下列叙述中正确的有 () 个

- I. 对 N 个记录进行快速排序, 在最坏情况下其时间复杂度退化为 $O(N^2)$, 此时其空间复杂度依然可以保持为 $O(\log_2 N)$
- II. 堆排序在最好、最坏和平均情况下的时间复杂度均为 $O(N \log_2 N)$, 且其在建堆 (Build-Heap) 阶段所需的时间复杂度为 $O(N)$
- III. 若初始序列完全逆序, 则起泡排序 (bubble sort) 和直接插入排序所需的比较次数和移动次数均达到最大值
- IV. 希尔排序 (shell sort) 的最后一趟实际上就是增量为 1 的直接插入排序, 因此希尔排序也是一种稳定的排序算法

A. 1

B. 2

C. 3

D. 4

正确答案: B

详细解析:

I 错误 (经典陷阱): 快速排序在最坏情况下 (如初始序列已经有序或逆序, 且每次选择第一个元素作为枢轴), 递归树会退化为单链表。此时递归调用栈的深度达到最大, 空间复杂度会随之退化为 $O(N)$, 而不是 $O(\log_2 N)$

II 正确: 堆排序的性能极其稳定, 无论数据初始状态如何, 其时间复杂度均稳定在 $O(N \log_2 N)$ 。并且, 从下往上建初始堆的过程中, 由于大部分节点位于树的底层, 向下调整的路径很短, 严格的数学推导证明其建堆时间复杂度为 $O(N)$

III 正确: 当序列完全逆序时, 起泡排序和直接插入排序每次比较都必须伴随着元素的移动 (交换), 此时这两者的比较次数和移动次数都达到了理论上的最大值 $O(N^2)$

IV 错误: 虽然希尔排序的最后一趟确实是直接插入排序, 但在之前的宏观增量分组排序中, 相同的元素可能会被分到不同的组中从而发生跨越式交换, 导致相对位置改变。因此, 希尔排序是不稳定的

27. 【考点: 简单选择排序、二路归并排序】以下关于比较类排序算法运行过程的说法中, 正确的是 ()

- A. 二路归并排序 (merge sort) 在合并两个长度分别为 m 和 n 的有序表时, 最坏情况下的比较次数为 $m + n$, 最好情况下的比较次数为 $\min(m, n)$
- B. 在简单选择排序中, 第一趟排序结束后, 最小 (或最大) 的元素一定已经到达其最终位置, 且任何情况下该算法的总比较次数均为 $N(N-1)/2$
- C. 判断一个序列是否为某趟排序的中间结果, 若发现序列的前半部分已经完全有序, 且后半部分元素均大于前半部分, 则该状态必定是由快速排序产生的
- D. 若使用基于比较的算法对 N 个元素进行排序, 只要算法设计得足够优秀, 其最坏情况下的时间复杂度下界可以突破 $O(N \log_2 N)$, 达到 $O(N)$

正确答案：B

详细解析：

A 错误：合并两个有序表时，最坏情况下的比较次数应为 $m + n - 1$ （当两个表的元素交错且最后一个元素直接落下时）；最好情况下的比较次数不是 $\min(m, n)$ ，而是 $\min(m, n)$ 次比较后，较短的表先空了，或者一个表的所有元素都小于另一个表

B 正确：简单选择排序的特性是“无论初始序列状态如何，比较次数恒定”。它通过 $N-1$ 趟选出最小值，总比较次数固定为 $\sum_{i=1}^{N-1} i = N(N-1) / 2$

C 错误：虽然快速排序的一趟划分会将序列分为两部分且存在枢轴，但“前半部分完全有序”并不是快速排序一趟结束的必然特征。这也有可能是堆排序在进行了几次提取最大值后，或是直接插入排序处理到中途的偶然状态

D 错误：基于比较的排序算法（如快速排序、归并排序、堆排序），可以通过决策树模型证明，其在最坏和平均情况下的时间复杂度理论下界就是 $O(N \log_2 N)$ ，无法突破。能达到 $O(N)$ 的排序（如基数排序）都不属于基于比较的排序

28. 【考点：外部排序】在进行外部排序时，为了优化磁盘 I/O 并提高整体排序效率，通常会引入多路归并、败者树（Loser Tree）以及置换-选择排序算法。关于这些技术的协同作用，下列推论中正确的是（ ）

I. 引入败者树的主要目的是为了减少多路归并时的磁盘读写次数

II. 采用置换-选择排序算法生成的初始归并段长度是可变的，其平均长度通常约为内存工作区大小（W）的 2 倍，这能够有效减少初始归并段的数量

III. 增加归并路数 k 可以减少归并趟数 S，因此在可用内存总量固定的情况下，盲目增大 k 值必定能使得整个外部排序的总时间大幅缩短

IV. 在进行 k 路严格树形归并时，若初始归并段数量为 M，且 $(M - 1) \pmod{k - 1} \neq 0$ ，则必须添加长度为 0 的“虚段”才能构造出严格的 k 叉树

A. 仅 I、II

B. 仅 II、IV

C. 仅 II、III、IV

D. I、II、IV

正确答案：B

详细解析：

I 错误：败者树的作用仅仅是减少在内存中进行 k 路比较的次数（将内部比较时间从 $O(k)$ 降至 $O(\log_2 k)$ ），它绝对不能减少磁盘 I/O 次数。磁盘 I/O 次数是由归并趟数 S 决定的

II 正确：置换-选择排序通过在内存中动态替换元素，可以生成长度大于内存工作区 W 的归并段，数学期望证明其平均长度约为 2W。初始归并段变长，数量 M 就会减少，从而减少后续的归并趟数

III 错误：这是一个经典的制约关系。虽然增大 k 能减少归并趟数，但如果“内存总量固定”，增大 k 意味着分配给每条输入缓冲区的内存变小了。缓冲区变小会导致磁盘每次连续读取的数据量下降，从而导致读写磁盘的次数（寻道时间）急剧增加，反而可能拖慢总时间

IV 正确：外部多路归并可以看作是一棵严格的 k 叉树，除了叶子节点（初始归并段），每个非叶子节点都必须恰好有 k 个分支。每进行一次 k 路归并，归并段数量减少 k-1 个。为了让最终归并为一个段，初始归并段加上补充的虚段总量必须满足 $(M_{\text{total}} - 1) \pmod{k - 1} = 0$ 。如果不满足，就必须填充虚段

29. 1. 算法设计思想

本题若采用常规的比较排序（如堆排序、二路归并排序等），时间复杂度最快为 $O(n \log_2 n)$ 。但利用待排元素仅有三种取值这一特殊性质，可以借鉴快速排序中的划分（Partition）操作来实现更高效的重排

具体设计思想如下（三指针法）：

设置三个指针：p0、p2 和 curr

p0 用于指示 0 的存放区域的右边界（初始化为 0）

p2 用于指示 2 的存放区域的左边界（初始化为 n-1）

curr 用于遍历数组（初始化为 0）

在 curr <= p2 的条件下，依次考察 A[curr] 的值：

若 A[curr] == 0：说明该元素应该在最前面。将其与 A[p0] 交换，然后将 p0 和 curr 均向后移动一位（p0++，curr++）

若 A[curr] == 1：说明该元素应该在中间。位置正确，直接将 curr 向后移动一位（curr++）

若 A[curr] == 2：说明该元素应该在最后面。将其与 A[p2] 交换，然后将 p2 向前移动一位（p2--）。注意：此时交换到 curr 位置的新元素尚未被考察，因此 curr 指针不能移动，必须在下一轮继续判断交换过来的元素

当 curr > p2 时，所有元素均已划分完毕，算法结束

2. C/C++ 代码实现

```
// 辅助函数：交换两个整数的值
void Swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// 核心排序算法：基于快速排序思想的三向切分
void SortColors(int A[], int n) {
    int p0 = 0;           // 指向存放 0 的区域的下一个可用位置
    int p2 = n - 1;       // 指向存放 2 的区域的前一个可用位置
    int curr = 0;         // 当前遍历指针

    // 当 curr 越过 p2 时，说明所有未处理的元素都已经处理完毕
    while (curr <= p2) {
        if (A[curr] == 0) {
            // 如果当前元素是 0，换到前面的 p0 位置
            Swap(&A[curr], &A[p0]);
            p0++;           // 0 的区域扩大
            curr++;         // 换过来的肯定是 1（或自己换自己），可以继续考察下一个元素
        }
        else if (A[curr] == 1) {
            // 如果当前元素是 1，直接跳过
            curr++;
        }
        else if (A[curr] == 2) {
            // 如果当前元素是 2，换到后面的 p2 位置
            Swap(&A[curr], &A[p2]);
            p2--;           // 2 的区域扩大
            // 注意：此时不能 curr++，因为从后面换过来的元素还没有被考察过
        }
    }
}
```

3. 复杂度分析

时间复杂度：算法通过 curr 指针一次性扫描整个数组。在每一次循环中，要么 curr 加 1，要么 p2 减 1。因此，总的交换和比较次数不会超过 n 次。时间复杂度为 O(n)

空间复杂度：算法完全在原数组上进行操作，仅使用了 `p0`、`p2`、`curr` 以及交换函数中的 `temp` 这几个常数级的辅助变量。空间复杂度为 $O(1)$

30. 直接插入排序 (Direct Insertion Sort)

```
// A[1...n] 存放数据, A[0] 作为哨兵
void InsertSort(int A[], int n) {
    int i, j;
    for (i = 2; i <= n; i++) {           // 依次将 A[2]~A[n] 插入到前面已排序序列
        if (A[i] < A[i - 1]) {           // 若 A[i] 小于其前驱, 需将 A[i] 插入有序
表
            A[0] = A[i];                 // 复制为哨兵, A[0] 暂存
            for (j = i - 1; A[0] < A[j]; --j) { // 从后往前查找插入位置
                A[j + 1] = A[j];         // 向后挪位
            }
            A[j + 1] = A[0];             // 复制到插入位置
        }
    }
}
```

折半插入排序 (Binary Insertion Sort)

```
// A[1...n] 存放数据, A[0] 暂存元素
void InsertSort_Binary(int A[], int n) {
    int i, j, low, high, mid;
    for (i = 2; i <= n; i++) {
        A[0] = A[i];                     // 暂存待插入元素
        low = 1; high = i - 1;          // 折半查找的范围
        while (low <= high) {            // 折半查找 (寻找插入位置)
            mid = low + (high - low) / 2;
            if (A[mid] > A[0]) {
                high = mid - 1;          // 插入点在左半区
            } else {
                low = mid + 1;           // 插入点在右半区 (保证算法稳定性)
            }
        }
        for (j = i - 1; j >= high + 1; --j) {
            A[j + 1] = A[j];            // 后移元素, 空出位置
        }
        A[high + 1] = A[0];             // 插入到正确位置
    }
}
```

希尔排序 (Shell Sort)

```
// A[1...n] 存放数据, A[0] 暂存元素
void ShellSort(int A[], int n) {
    int dk, i, j;
    // 步长 dk 依次减半, 直到为 1
    for (dk = n / 2; dk >= 1; dk = dk / 2) {
        for (i = dk + 1; i <= n; ++i) {
            if (A[i] < A[i - dk]) {      // 需将 A[i] 插入有序增量子表
                A[0] = A[i];             // 暂存在 A[0]
                for (j = i - dk; j > 0 && A[0] < A[j]; j -= dk) {
                    A[j + dk] = A[j];    // 记录后移, 查找插入位置
                }
            }
        }
    }
}
```



```

        A[j + dk] = A[0];    // 插入
    }
}
}
}

```

冒泡排序 (Bubble Sort)

```

void BubbleSort(int A[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool flag = false;    // 标记本趟冒泡是否发生交换
        for (int j = n - 1; j > i; j--) { // 一趟冒泡，从后往前
            if (A[j - 1] > A[j]) {
                int temp = A[j - 1];
                A[j - 1] = A[j];
                A[j] = temp;    // 交换
                flag = true;
            }
        }
        if (flag == false) return;    // 若未发生交换，说明序列已然有序，提前结束
    }
}

```

快速排序 (Quick Sort)

```

// 核心划分函数 (Partition)
int Partition(int A[], int low, int high) {
    int pivot = A[low];    // 第一个元素作为枢轴
    while (low < high) {
        while (low < high && A[high] >= pivot) --high;
        A[low] = A[high];    // 比枢轴小的移到左端
        while (low < high && A[low] <= pivot) ++low;
        A[high] = A[low];    // 比枢轴大的移到右端
    }
    A[low] = pivot;    // 枢轴元素归位
    return low;    // 返回枢轴最终位置
}

// 快速排序递归主体
void QuickSort(int A[], int low, int high) {
    if (low < high) {
        int pivotpos = Partition(A, low, high); // 划分
        QuickSort(A, low, pivotpos - 1);    // 递归左子表
        QuickSort(A, pivotpos + 1, high);    // 递归右子表
    }
}

```

简单选择排序 (Simple Selection Sort)

```

void SelectSort(int A[], int n) {
    for (int i = 0; i < n - 1; i++) { // 一共进行 n-1 趟
        int min = i;    // 记录最小元素的位置
        for (int j = i + 1; j < n; j++) {
            if (A[j] < A[min]) {
                min = j;    // 更新最小元素位置
            }
        }
    }
}

```

```

    }
    if (min != i) { // 将最小元素换至边界
        int temp = A[i];
        A[i] = A[min];
        A[min] = temp;
    }
}
}

```

堆排序 (Heap Sort)

```

// 核心：向下调整函数（以大顶堆为例）
void HeadAdjust(int A[], int k, int len) {
    A[0] = A[k]; // A[0] 暂存筛选节点
    for (int i = 2 * k; i <= len; i *= 2) { // 沿 key 较大的子节点向下筛选
        if (i < len && A[i] < A[i + 1]) {
            i++; // 左右孩子中取较大者
        }
        if (A[0] >= A[i]) {
            break; // 筛选结束
        } else {
            A[k] = A[i]; // 将 A[i] 调整到双亲结点上
            k = i; // 修改 k 值，以便继续向下筛选
        }
    }
    A[k] = A[0]; // 被筛选结点的值放入最终位置
}

// 建立大顶堆
void BuildMaxHeap(int A[], int len) {
    for (int i = len / 2; i > 0; i--) { // 从最后一个非叶子结点开始往上调整
        HeadAdjust(A, i, len);
    }
}

// 堆排序主函数
void HeapSort(int A[], int len) {
    BuildMaxHeap(A, len); // 1. 建立初始堆
    for (int i = len; i > 1; i--) { // 2. 进行 n-1 次输出和调整
        int temp = A[i]; // 堆顶 A[1] 与当前堆底 A[i] 交换
        A[i] = A[1];
        A[1] = temp;
        HeadAdjust(A, 1, i - 1); // 调整剩余 i-1 个元素，使其重新成堆
    }
}

```

归并排序 (Merge Sort)

```

#define MaxSize 1000
int B[MaxSize]; // 辅助数组 B

void Merge(int A[], int low, int mid, int high) {
    int i, j, k;
    for (k = low; k <= high; k++) {
        B[k] = A[k]; // 将 A 中所有元素复制到 B 中
    }
}

```

```

        for (i = low, j = mid + 1, k = i; i <= mid && j <= high; k++) {
            if (B[i] <= B[j]) {                // 比较 B 的左右两段
                A[k] = B[i++];                // 将较小值复制回 A 中
            } else {
                A[k] = B[j++];
            }
        }
        while (i <= mid) A[k++] = B[i++]; // 若左半区未拷完, 直接拷回 A
        while (j <= high) A[k++] = B[j++]; // 若右半区未拷完, 直接拷回 A
    }

void MergeSort(int A[], int low, int high) {
    if (low < high) {
        int mid = low + (high - low) / 2; // 划分
        MergeSort(A, low, mid);          // 递归排序左半区
        MergeSort(A, mid + 1, high);      // 递归排序右半区
        Merge(A, low, mid, high);         // 归并
    }
}

```

计数排序 (Counting Sort)

```

void CountingSort(int A[], int n) {
    if (n <= 1) return;

    // 1. 寻找数组中的最大值和最小值
    int max = A[0], min = A[0];
    for (int i = 1; i < n; i++) {
        if (A[i] > max) max = A[i];
        if (A[i] < min) min = A[i];
    }

    int range = max - min + 1;
    int *count = (int *)calloc(range, sizeof(int)); // 计数数组, 初始化为0
    int *output = (int *)malloc(n * sizeof(int));   // 临时存储排序结果

    // 2. 统计每个元素出现的次数
    for (int i = 0; i < n; i++) {
        count[A[i] - min]++;
    }

    // 3. 累加次数 (累加后, count[x] 表示小于等于 x 的元素个数)
    for (int i = 1; i < range; i++) {
        count[i] += count[i - 1];
    }

    // 4. 反向遍历原数组 (确保排序稳定性), 将元素放入对应位置
    for (int i = n - 1; i >= 0; i--) {
        output[count[A[i] - min] - 1] = A[i];
        count[A[i] - min]--;           // 放置一个, 位置往前挪一个
    }

    // 5. 复制回原数组 A
    for (int i = 0; i < n; i++) {
        A[i] = output[i];
    }
}

```

```
    free(count);  
    free(output);  
}
```